Decision Trees

```
# ID3 Simple
import pandas as pd
import numpy as np
```

```
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain',
'Overcast', 'Sunny', 'Sunny', 'Rain', 'Sunny', 'Overcast', 'Overcast',
'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Mild',
'Mild', 'Hot', 'Mild', 'Cool', 'Mild', 'Cool', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal',
'Normal', 'High', 'Normal', 'Normal', 'High', 'Normal', 'Normal', 'High'],
    'Windy': [False, True, False, False, False, True, True, False, False,
False, True, True, False, True],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes',
'Yes', 'Yes', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)
df
```

Output of the code:-

	Outlook	Temperature	Humidity	Windy	Play
0	Sunny	Hot	High	False	No
1	Sunny	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Rain	Mild	High	False	Yes
4	Rain	Cool	Normal	False	Yes
5	Rain	Cool	Normal	True	No
6	Overcast	Mild	Normal	True	Yes
7	Sunny	Mild	High	False	No
8	Sunny	Hot	Normal	False	Yes
9	Rain	Mild	Normal	False	Yes
10	Sunny	Cool	High	True	Yes
11	Overcast	Mild	Normal	True	Yes
12	Overcast	Cool	Normal	False	Yes
13	Rain	Mild	High	True	No

```
def entropy(target_col):
    elements, counts = np.unique(target_col, return_counts=True)
    entropy = -sum((counts / counts.sum()) * np.log2(counts /
counts.sum()))
    return entropy
```

```
def information_gain(data, split_attribute_name, target_name):
    # Calculate the total entropy for the target attribute
    total_entropy = entropy(data[target_name])

    # Find the unique values for the split attribute
    values, counts = np.unique(data[split_attribute_name],
return_counts=True)

    # Calculate the weighted entropy for each value
```

```
    weighted_entropy = sum((counts[i] / np.sum(counts)) *
entropy(data.where(data[split_attribute_name] ==
values[i]).dropna()[target_name]) for i in range(len(values)))

    # Calculate the information gain
    return total_entropy - weighted_entropy
```

```
def build_tree(data, target_name):
    # Get the unique values for the target attribute
    unique_classes, counts = np.unique(data[target_name],
return_counts=True)

    # If all target values are the same, return that value
    if len(unique_classes) == 1:
        return unique_classes[0]

    # If there are no more features to split on, return the most common
target value
    if len(data.columns) == 1:  # Only target column left
        return unique_classes[np.argmax(counts)]

    # Calculate information gain for each attribute
    gains = [information_gain(data, attr, target_name) for attr in
data.columns if attr != target_name]

    # Find the attribute with the highest information gain
    best_attr_index = np.argmax(gains)
    best_attr = data.columns[best_attr_index]

    # Create a tree node
    tree = {best_attr: {}}

    # Split the dataset on the best attribute
    for value in data[best_attr].unique():
        sub_data = data[data[best_attr] == value]
        # Recursively build the tree for each split
        subtree = build_tree(sub_data.drop(columns=[best_attr]),
target_name)
        tree[best_attr][value] = subtree
```

```
    return tree
```

```
tree = build_tree(df, 'Play')
print(tree)
```

Output of the code:-

{'Outlook': {'Overcast': 'Yes', 'Rain': {'Windy': {np.False_: 'Yes', np.True_: 'No'}}, 'Sunny': {'Temperature': {'Cool': 'Yes', 'Hot': {'Humidity': {'High': 'No', 'Normal': 'Yes'}}, 'Mild': 'No'}}}}

```
# ============================================================
# ID3 DECISION TREE ALGORITHM - Detail implementation
# PLAY TENNIS DATASET
# ============================================================

# Step 1: Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# Step 2: Create Play Tennis Dataset
# ============================================================

data = {
    'Outlook': [ 'Sunny', 'Sunny', 'Overcast', 'Rain','Rain', 'Rain',
'Overcast', 'Sunny','Sunny', 'Rain', 'Sunny', 'Overcast','Overcast',
'Rain'],

    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild','Cool',
'Cool','Mild','Mild','Hot', 'Mild','Cool','Mild','Cool','Mild' ],

    'Humidity': ['High', 'High', 'High', 'High','Normal','Normal',
'Normal','High','Normal','Normal','High','Normal','Normal','High' ],
```

```
    'Windy': [False, True, False, False,False, True, True, False,False,
False, True, True,False, True ],

    'Play': ['No', 'No', 'Yes', 'Yes','Yes', 'No', 'Yes', 'No','Yes',
'Yes', 'Yes', 'Yes','Yes', 'No' ]
}


# Convert dictionary into DataFrame
df = pd.DataFrame(data)
# Display Dataset
print("\n================ PLAY TENNIS DATASET ================\n")
print(df)

# ============================================================
# Step 3: Calculate Entropy
# ============================================================

def entropy(target_col):

    # Find unique classes and their counts
    elements, counts = np.unique(
        target_col,
        return_counts=True
    )

    # Calculate probabilities
    probabilities = counts / counts.sum()

    # Calculate entropy
    entropy_value = -np.sum(
        probabilities * np.log2(probabilities)
    )
    return entropy_value

# ============================================================
# Step 4: Calculate Information Gain
# ============================================================

def information_gain(data, attribute, target):
```

```
    # Entropy of complete dataset
    total_entropy = entropy(data[target])

    # Find unique values of the attribute
    values, counts = np.unique( data[attribute],return_counts=True)

    # Initialize weighted entropy
    weighted_entropy = 0

    # Calculate weighted entropy
    for value, count in zip(values, counts):

        # Select subset
        subset = data[data[attribute] == value]

        # Entropy of subset
        subset_entropy = entropy(subset[target])

        # Weight of subset
        weight = count / len(data)

        # Add weighted entropy
        weighted_entropy += (weight * subset_entropy)

    # Information Gain
    gain = total_entropy - weighted_entropy

    return gain


# ============================================================
# Step 5: Display Entropy of Dataset
# ============================================================

print("\n================ ENTROPY ================\n")

dataset_entropy = entropy(df['Play'])

print("Entropy of Play =",round(dataset_entropy, 4))
```

```
# ============================================================
# Step 6: Calculate Information Gain for All Attributes
# ============================================================

print("\n=========== INFORMATION GAIN =============\n")

attributes = [
    column for column in df.columns
    if column != 'Play' ]

for attribute in attributes:

    gain = information_gain(df,attribute,'Play')

    print(f"{attribute:12s} = {gain:.4f}")

# ============================================================
# Step 7: Build ID3 Decision Tree
# ============================================================

def build_tree(data, target):

    # ---------------------------------------------------------
    # Find unique classes and their counts
    # ---------------------------------------------------------

    classes, counts = np.unique(data[target],return_counts=True)

    # ---------------------------------------------------------
    # Stopping Condition 1
    # All records belong to same class
    # ---------------------------------------------------------

    if len(classes) == 1:
        return classes[0]

    # ---------------------------------------------------------
    # Get predictor attributes
    # ---------------------------------------------------------
```

```
    attributes = [
        column for column in data.columns
        if column != target
    ]


    # ----------------------------------------------------------
    # Stopping Condition 2
    # No attributes left
    # Return majority class
    # ----------------------------------------------------------

    if len(attributes) == 0:

        return classes[np.argmax(counts)]

    # ----------------------------------------------------------
    # Calculate Information Gain
    # ----------------------------------------------------------
    gains = []

    for attribute in attributes:
        gain = information_gain(data,attribute,target)

        gains.append(gain)


    # ----------------------------------------------------------
    # Select attribute with maximum Information Gain
    # ----------------------------------------------------------

    best_attribute = attributes[np.argmax(gains)]
    # ----------------------------------------------------------
    # Create tree node
    # ----------------------------------------------------------
    tree = { best_attribute: {} }

   # ----------------------------------------------------------
    # Split data using best attribute
```

```
    # ----------------------------------------------------

    values = np.unique(data[best_attribute])

    for value in values:
        # Create subset
        subset = data[data[best_attribute] == value].copy()
        # Remove selected attribute
        subset = subset.drop(columns=[best_attribute])

        # Recursively build subtree
        subtree = build_tree(subset,target)

        # Add subtree to tree
        tree[best_attribute][value] = subtree

    return tree
# ============================================================
# Step 8: Build the ID3 Tree
# ============================================================

tree = build_tree(df,'Play')

# ============================================================
# Step 9: Print Tree as Dictionary
# ============================================================

print("\n================ ID3 TREE ================\n")
print(tree)
# ============================================================
# Step 10: Display Tree in Text Format
# ============================================================

def display_tree(tree, indent=""):

    # If leaf node
    if not isinstance(tree, dict):

        print( indent + "└── " + str(tree))
```

```
        return


    # Get attribute
    attribute = list(tree.keys())[0]

    print(indent + attribute)

    # Get branches
    branches = tree[attribute]

    branch_items = list(branches.items())


    for index, (value, subtree) in enumerate(branch_items):

        last_branch = (index == len(branch_items) - 1)


        if last_branch:

            symbol = "└── "
            next_indent = indent + "    "

        else:

            symbol = "├── "
            next_indent = indent + "│   "


        print(indent +symbol +str(value))


        if isinstance(subtree, dict):

            display_tree(subtree,next_indent)

        else:

            print(next_indent +"└── " +str(subtree)
```

```
            )

print("\n============= TREE STRUCTURE =============\n")

display_tree(tree)

# ============================================================
# Step 11: Get Node Information
# ============================================================

def get_node_information(data,target,class_names):

    # Number of samples
    samples = len(data)

    # Class distribution
    class_counts = []

    for class_name in class_names:

        count = np.sum(data[target] == class_name)

        class_counts.append(int(count))
    # Entropy
    node_entropy = entropy(data[target])

    # Majority class
    predicted_class = class_names[np.argmax(class_counts)]

    return (node_entropy,samples,class_counts, predicted_class)


# ============================================================
# Step 12: Count Number of Leaf Nodes
# ============================================================

def count_leaves(tree):

    # Leaf node
    if not isinstance(tree, dict):
```

```
        return 1

    total = 0

    for branches in tree.values():

        for subtree in branches.values():

            total += count_leaves(subtree)

    return total
# ============================================================
# Step 13: Calculate Tree Depth
# ============================================================

def tree_depth(tree):

    # Leaf node
    if not isinstance(tree, dict):

        return 1

    depths = []

    for branches in tree.values():

        for subtree in branches.values():

            depths.append(tree_depth(subtree))

    return 1 + max(depths)

# ============================================================
# Step 14: Draw ID3 Tree
# ============================================================

def draw_tree(tree,data,target,class_names,ax,x,y,x_spacing,y_spacing,
        parent_x=None,parent_y=None,edge_label=None):
```

```
    # ------------------------------------------------------------
    # Get node information
    # ------------------------------------------------------------

    node_entropy, samples, class_counts, predicted_class = \
        get_node_information( data,target,class_names )

    # ------------------------------------------------------------
    # Create node text
    # ------------------------------------------------------------

    if isinstance(tree, dict):

        attribute = list(tree.keys())[0]

        node_text = (
            f"{attribute}\n"
            f"entropy = {node_entropy:.3f}\n"
            f"samples = {samples}\n"
            f"value = {class_counts}\n"
            f"class = {predicted_class}"
        )

    else:

        node_text = (
            f"entropy = {node_entropy:.3f}\n"
            f"samples = {samples}\n"
            f"value = {class_counts}\n"
            f"class = {predicted_class}"
        )


    # ------------------------------------------------------------
    # Select node color
    # ------------------------------------------------------------

    if predicted_class == "Yes":

        node_color = "#90EE90"
```

```
    elif predicted_class == "No":

        node_color = "#F4A460"

    else:

        node_color = "#ADD8E6"

    # ------------------------------------------------------------
    # Draw line from parent
    # ------------------------------------------------------------

    if parent_x is not None:

        ax.plot(
            [parent_x, x],
            [parent_y - 0.08, y + 0.08],
            linewidth=1.2,
            color="black"
        )


        # Branch label
        mid_x = (parent_x + x) / 2

        mid_y = (parent_y + y) / 2



ax.text(mid_x,mid_y,str(edge_label),fontsize=11,ha="center",va="center",
            bbox=dict(facecolor="white",edgecolor="none",pad=1)
        )
    # ------------------------------------------------------------
    # Draw node
    # ------------------------------------------------------------

    ax.text(
        x,
        y,
```

```
        node_text,
        fontsize=11,
        ha="center",
        va="center",
        bbox=dict(
            boxstyle="round,pad=0.5",
            facecolor=node_color,
            edgecolor="black",
            linewidth=1.2
        )
    )

    # ------------------------------------------------------------
    # Stop if leaf
    # ------------------------------------------------------------

    if not isinstance(tree, dict):

        return
    # ------------------------------------------------------------
    # Get attribute and branches
    # ------------------------------------------------------------

    attribute = list(tree.keys() )[0]

    branches = tree[attribute]

    # ------------------------------------------------------------
    # Calculate child positions
    # ------------------------------------------------------------

    number_of_children = len(branches)

    start_x = (x -((number_of_children - 1)*x_spacing / 2))

    # ------------------------------------------------------------
    # Draw children
    # ------------------------------------------------------------

    for index, (value, subtree) in enumerate(
```

```
        branches.items()
    ):

        child_x = (start_x +index * x_spacing)

        child_y = (y - y_spacing)

        # Data for current branch
        child_data = data[data[attribute] == value].copy()

        # Recursive call
        draw_tree(
            subtree,
            child_data,
            target,
            class_names,
            ax,
            child_x,
            child_y,
            x_spacing / 2,
            y_spacing,
            x,
            y,
            value
        )

# ============================================================
# Step 15: Generate Graphical Decision Tree
# ============================================================

# Get class names
class_names = sorted(df['Play'].unique())

# Number of leaves
number_of_leaves = count_leaves(tree)

# Tree depth
depth = tree_depth(tree)

# Figure size
```

```
figure_width = max(14,number_of_leaves * 3)

figure_height = max(8,depth * 3)

# Create figure
fig, ax = plt.subplots(figsize=(figure_width,figure_height))

# Draw tree
draw_tree(
    tree=tree,
    data=df,
    target='Play',
    class_names=class_names,
    ax=ax,
    x=0,
    y=0,
    x_spacing=10,
    y_spacing=3,
    parent_x=None,
    parent_y=None,
    edge_label=None
)

# Remove axes
ax.axis('off')


# Add title
plt.title("ID3 Decision Tree - Play Tennis Dataset",
fontsize=18,fontweight='bold')

# Adjust layout
plt.tight_layout()

# Display tree
plt.show()
```

Output of the code:-

========== PLAY TENNIS DATASET ==========

```
    Outlook Temperature Humidity  Windy Play
0     Sunny       Hot    High  False   No
1     Sunny       Hot    High   True   No
2  Overcast       Hot    High  False  Yes
3      Rain      Mild    High  False  Yes
4      Rain      Cool   Normal  False  Yes
5      Rain      Cool   Normal   True   No
6  Overcast      Mild   Normal   True  Yes
7     Sunny      Mild    High  False   No
8     Sunny       Hot   Normal  False  Yes
9      Rain      Mild   Normal  False  Yes
10    Sunny      Cool    High   True  Yes
11 Overcast      Mild   Normal   True  Yes
12 Overcast      Cool   Normal  False  Yes
13     Rain      Mild    High   True   No

================ ENTROPY ================

Entropy of Play = 0.9403

=========== INFORMATION GAIN =============

Outlook     = 0.2467
Temperature  = 0.0292
Humidity    = 0.1518
Windy       = 0.0481

================ ID3 TREE ================

{'Outlook': {'Overcast': 'Yes', 'Rain': {'Windy': {np.False_: 'Yes', np.True_: 'No'}},
'Sunny': {'Temperature': {'Cool': 'Yes', 'Hot': {'Humidity': {'High': 'No', 'Normal':
'Yes'}}, 'Mild': 'No'}}}}

============= TREE STRUCTURE =============

Outlook
├─Overcast
│  └─Yes
├─Rain
│  Windy
│  ├─False
│  │  └─Yes
```

```
│  └─True
│     └─No
└─Sunny
  Temperature
   ├─Cool
   │  └─Yes
   ├─Hot
   │  Humidity
   │  ├─High
   │  │  └─No
   │  └─Normal
   │     └─Yes
   └─Mild
      └─No
```

